

SQL

SQL, Structured Query Language, is the language for working with data in a relational database. We describe what we want, and the database decides how to fetch it.

A reading order:

The table → Keyword groups → Datatypes → Operators → Aggregate queries

1. The table we will use

Every example below starts from this one table, as it is shown here. `CREATE TABLE` defines the columns and the type of each one.

```
CREATE TABLE employees (  
  id          INT PRIMARY KEY,  
  name        VARCHAR(50) NOT NULL,  
  department  VARCHAR(30),  
  salary      DECIMAL(10, 2),  
  hired_on    DATE  
);
```

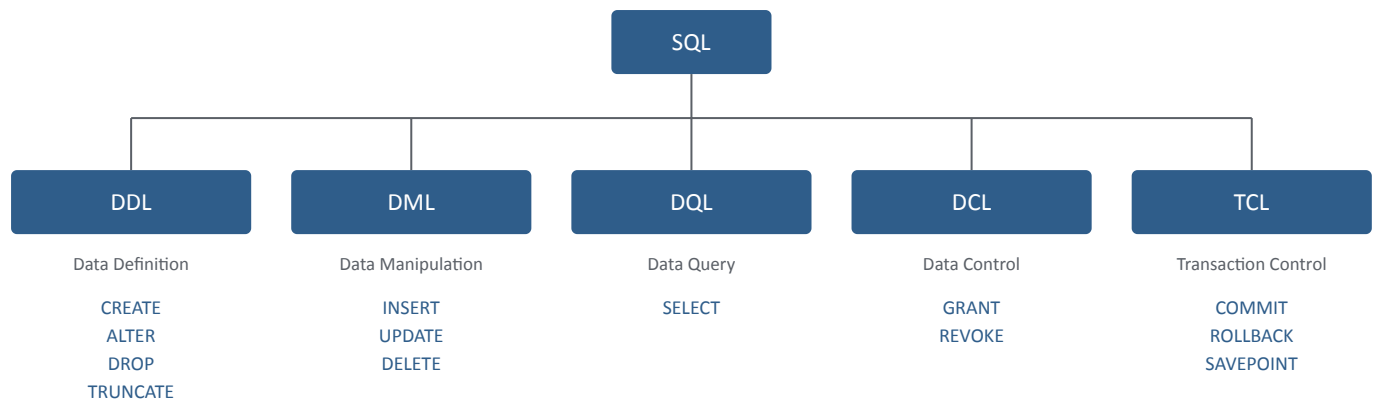
These are the rows it holds:

id	name	department	salary	hired_on
1	Ravi	Engineering	75000.00	2020-06-15
2	Amit	Engineering	68000.00	2021-03-01
3	Priya	Sales	59000.00	2019-11-20
4	Neha	Sales	62000.00	2022-08-05
5	Arjun	Marketing	54000.00	2023-01-10
6	Kiran	NULL	48000.00	2024-02-14

Kiran has no department. That `NULL` is used later to show how SQL treats a missing value.

2. Keyword groups

SQL keywords are reserved words the database already understands. They split into five groups, and naming the group a statement belongs to is a common interview question.



DDL changes the shape of the data. DML changes the data itself. DQL only reads it.

Group	Changes	Can it be rolled back
DDL	The structure: tables and columns	No, it commits on its own
DML	The rows inside a table	Yes
DQL	Nothing. It only reads	Nothing to roll back
DCL	Who is allowed to do what	No
TCL	Whether recent DML is kept or undone	It is the mechanism

DDL, Data Definition Language

DDL changes the shape of the database, not the data in it.

Keyword	What it does
CREATE	Makes a new table
ALTER	Changes a table that already exists
TRUNCATE	Removes every row and keeps the table
DROP	Removes the table itself

```

-- CREATE: a new table
CREATE TABLE departments (
    id    INT PRIMARY KEY,
    name  VARCHAR(30) NOT NULL
);

-- ALTER: add a column
ALTER TABLE employees ADD COLUMN email VARCHAR(100);

-- ALTER: remove a column
ALTER TABLE employees DROP COLUMN email;

-- ALTER: rename the table
ALTER TABLE departments RENAME TO teams;

-- TRUNCATE: every row goes, the table stays
TRUNCATE TABLE teams;

-- DROP: remove the table itself
DROP TABLE teams;

```

After `TRUNCATE` the table is still there and `SELECT` returns nothing. After `DROP` the same `SELECT` fails, because the table no longer exists.

Changing a column's type is the one piece of `ALTER` that is written differently in each database: MySQL uses `MODIFY COLUMN`, PostgreSQL uses `ALTER COLUMN ... TYPE`.

DML, Data Manipulation Language

DML changes the rows.

Keyword	What it does
<code>INSERT</code>	Adds new rows
<code>UPDATE</code>	Changes rows that exist
<code>DELETE</code>	Removes rows

`INSERT` adds rows. Several go in one statement.

```

INSERT INTO employees (id, name, department, salary, hired_on)
VALUES
    (7, 'Sara', 'Engineering', 71000.00, '2024-05-02'),
    (8, 'Dev', 'Sales', 57000.00, '2024-06-11');

```

Output:

2 rows inserted

Name the columns even though it is longer to type. Without the list the values have to match the table's column order exactly, so adding a column later breaks every insert that left it out.

UPDATE changes rows that already exist. **SET** says which columns change, and **WHERE** says which rows.

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Sales';
```

Output:

2 rows updated

Those two rows now read:

id	name	department	salary
3	Priya	Sales	64900.00
4	Neha	Sales	68200.00

An **UPDATE** with no **WHERE** changes every row in the table. There is no undo unless we are inside a transaction.

```
UPDATE employees SET salary = 0; -- every employee, not one
```

A habit that prevents this: write the same filter as a **SELECT** first, check the rows that come back, then change the word **SELECT** to **UPDATE**.

DELETE removes rows. Like **UPDATE** it takes a **WHERE**, and without one it empties the table.

```
DELETE FROM employees
WHERE id = 6;
```

Output:

1 row deleted

Three statements remove data, and interviews ask for the difference:

	DELETE	TRUNCATE	DROP
Removes	Chosen rows	All rows	The whole table
Takes a <code>WHERE</code>	Yes	No	No
Table still exists	Yes	Yes	No
Group	DML	DDL	DDL
Can be rolled back	Yes	Usually not	Usually not
Speed on a large table	Slow, row by row	Fast	Fast

DELETE → some rows, undoable
 TRUNCATE → all rows, table stays
 DROP → table and rows both gone

DQL, Data Query Language

DQL is one keyword. It reads and changes nothing.

Keyword	What it does
<code>SELECT</code>	Reads rows

```

SELECT name, salary
FROM employees
WHERE department = 'Engineering';

```

Output:

```

name | salary
-----+-----
Ravi | 75000.00
Amit | 68000.00

```

`SELECT *` returns every column. It is convenient when exploring and a poor choice in application code, because the result changes whenever someone adds a column.

`ORDER BY` sorts the rows and `LIMIT` caps how many come back.

```
SELECT name, salary
FROM employees
WHERE salary > 60000
ORDER BY salary DESC
LIMIT 2;
```

Output:

```
name | salary
-----+-----
Ravi | 75000.00
Amit | 68000.00
```

`DESC` is highest first and `ASC` is lowest first. `ASC` is the default, so it is usually left out.

`DISTINCT` removes duplicate rows from the result.

```
SELECT DISTINCT department FROM employees WHERE department IS NOT NULL;
```

Output:

```
department
-----
Engineering
Marketing
Sales
```

DCL, Data Control Language

DCL decides who is allowed to do what. These are run by an administrator, not usually by an application.

Keyword	What it does
<code>GRANT</code>	Gives a user a permission
<code>REVOKE</code>	Takes a permission away

```
-- GRANT: let a user read and add rows
GRANT SELECT, INSERT ON employees TO reporting_user;

-- REVOKE: take one of those back
REVOKE INSERT ON employees FROM reporting_user;
```

`reporting_user` can now read `employees` and can no longer insert into it.

TCL, Transaction Control Language, is the fifth group: `COMMIT`, `ROLLBACK` and `SAVEPOINT`.

3. Datatypes

Every column has a type. The type decides what can be stored and how much space a row takes.

Type	Holds	Example
<code>INT</code>	A whole number	<code>42</code>
<code>BIGINT</code>	A larger whole number	<code>9876543210</code>
<code>DECIMAL(p, s)</code>	An exact decimal, <code>p</code> digits with <code>s</code> after the point	<code>75000.00</code>
<code>FLOAT</code> , <code>DOUBLE</code>	An approximate decimal	<code>3.14159</code>
<code>VARCHAR(n)</code>	Text up to <code>n</code> characters	<code>'Ravi'</code>
<code>CHAR(n)</code>	Text of exactly <code>n</code> characters, padded	<code>'IN'</code>
<code>TEXT</code>	Long text with no set limit	A description
<code>BOOLEAN</code>	<code>TRUE</code> or <code>FALSE</code>	<code>TRUE</code>
<code>DATE</code>	A date	<code>'2020-06-15'</code>
<code>TIME</code>	A time	<code>'09:30:00'</code>
<code>TIMESTAMP</code>	A date and a time	<code>'2020-06-15 09:30:00'</code>

Money → `DECIMAL`, never `FLOAT`
Short text → `VARCHAR(n)`
Long text → `TEXT`

`FLOAT` stores an approximation, so `0.1 + 0.2` does not equal exactly `0.3`. That is fine for a measurement and wrong for a salary, which is why `salary` is `DECIMAL(10, 2)`.

`VARCHAR(50)` uses only the space the text needs. `CHAR(50)` always uses fifty characters, padding the rest with spaces.

4. Operators

Operators compare and combine values, mostly inside `WHERE`.

Kind	Operators
Comparison	<code>=</code> , <code><></code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>
Logical	<code>AND</code> , <code>OR</code> , <code>NOT</code>

Kind	Operators
Range	<code>BETWEEN a AND b</code>
Set	<code>IN (...)</code> , <code>NOT IN (...)</code>
Pattern	<code>LIKE</code> , with <code>%</code> for any text and <code>_</code> for one character
Missing value	<code>IS NULL</code> , <code>IS NOT NULL</code>
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>

`<>` is not equal. Some databases also accept `!=`.

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 55000 AND 65000;
```

Output:

```
name | salary
-----+-----
Priya | 59000.00
Neha  | 62000.00
```

`LIKE` matches a pattern rather than an exact value:

```
SELECT name FROM employees WHERE name LIKE 'A%';
```

Output:

```
name
-----
Amit
Arjun
```

`NULL` is not a value, so `=` never matches it. `WHERE department = NULL` returns no rows, even for Kiran.

`IS NULL` is the only way to ask:

```
SELECT name FROM employees WHERE department IS NULL;
```

Output:

name

Kiran

5. Aggregate queries

An aggregate function reads many rows and returns one value.

Function	What it does
COUNT	Counts rows
SUM	Adds the values together
AVG	Returns the average
MIN	Returns the smallest value
MAX	Returns the largest value

Over the whole table:

```
SELECT COUNT(*) AS employees,  
       SUM(salary) AS total_salary,  
       AVG(salary) AS average_salary,  
       MIN(salary) AS lowest,  
       MAX(salary) AS highest  
FROM employees;
```

Output:

```
employees | total_salary | average_salary | lowest | highest  
-----+-----+-----+-----+-----  
6 | 366000.00 | 61000.00 | 48000.00 | 75000.00
```

Aggregates skip **NULL**, and **COUNT(*)** does not. This is the difference interviews ask about:

```
SELECT COUNT(*) AS all_rows,  
       COUNT(department) AS with_department  
FROM employees;
```

Output:

all_rows	with_department
6	5

`COUNT(*)` counts rows. `COUNT(department)` counts only the rows where `department` has a value, so Kiran is left out.

GROUP BY splits the rows into groups and runs the aggregate on each one.

```
SELECT department,
       COUNT(*)    AS headcount,
       AVG(salary) AS average_salary
FROM employees
WHERE department IS NOT NULL
GROUP BY department
ORDER BY department;
```

Output:

department	headcount	average_salary
Engineering	2	71500.00
Marketing	1	54000.00
Sales	2	60500.00

HAVING filters the groups. `WHERE` cannot do this, because it runs before the groups exist.

```
SELECT department, COUNT(*) AS headcount
FROM employees
WHERE department IS NOT NULL
GROUP BY department
HAVING COUNT(*) > 1
ORDER BY department;
```

Output:

department	headcount
Engineering	2
Sales	2

Marketing has one employee, so its group did not survive the `HAVING`.

WHERE → filters rows, before grouping
HAVING → filters groups, after grouping

The database applies the parts in a different order from the one we write them in:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

That order explains two rules. **HAVING** can use an aggregate and **WHERE** cannot. And every column in **SELECT** must either appear in **GROUP BY** or sit inside an aggregate, because anything else has no single value for the group:

```
-- Error: name is neither grouped nor aggregated
SELECT department, name, COUNT(*)
FROM employees
GROUP BY department;
```

COUNT(DISTINCT ...) counts the different values rather than the rows:

```
SELECT COUNT(DISTINCT department) AS departments FROM employees;
```

Output:

```
departments
-----
3
```

The statements together

Statement	Group	What it does	Needs a WHERE
SELECT	DQL	Reads rows	No, but usually has one
INSERT	DML	Adds new rows	No
UPDATE	DML	Changes existing rows	Almost always
DELETE	DML	Removes rows	Almost always
CREATE	DDL	Makes a table	No
ALTER	DDL	Changes a table	No
TRUNCATE	DDL	Empties a table	Cannot take one

Statement	Group	What it does	Needs a WHERE
DROP	DDL	Removes a table	Cannot take one
GRANT	DCL	Gives a permission	No
REVOKE	DCL	Takes a permission away	No
COMMIT	TCL	Keeps the transaction's work	No
ROLLBACK	TCL	Undoes the transaction's work	No
SAVEPOINT	TCL	Marks a point to roll back to	No

For example, imagine a payroll application. **CREATE TABLE** defines **employees** once. Each new hire is an **INSERT**. The yearly raise is an **UPDATE** with a **WHERE** on department, wrapped in a transaction so a mistake can be rolled back. The monthly report is a **SELECT** with **GROUP BY department** and an **AVG(salary)**. A record removed by request is a **DELETE** on one **id**.

Write a **SELECT** against a real table, then rewrite it as the **UPDATE** that changes exactly the rows it returned.